Chapter 15. Server-Side Java

15.0 Introduction

Sockets form the underpinnings of almost all network software implementations. HTTP(S), JDBC, RMI, CORBA, EJB, XMPP, and the non-Java Remote Procedure Call (RPC) and Network File System (NFS): all are implemented by connecting various types of sockets together. Socket connections can be implemented in most any language, not just Java: C (the original), C++, Perl, and Python are also popular, and many others are possible. A client or server written in any one of these languages can communicate with its opposite written in any of the other languages. Therefore, it's worth taking a quick look at how the `ServerSocket` behaves, even if you wind up utilizing one of the higher-level services.

The discussion looks first at the `ServerSocket` itself, then at writing data over a socket in various ways. Finally, I show a complete implementation of a usable network server written in Java: the chat server from the client in the previous chapter.

TIP

Most production work in server-side Java uses the Jakarta Enterprise Edition (Jakarta EE, formerly Java EE), transferred years ago from Oracle to the Eclipse Software Foundation and renamed to Jakarta, but still widely referred to by its previous name (and occasionally by its very old name, "J2EE," which was retired some *20 years ago*). Those that don't use EE directly use some of its core APIs buried in other frameworks, like Spring Boot, Quarkus, Helidon, and more. Jakarta EE provides scalability and support for building well-structured, multitiered distributed applications. EE provides the servlet framework; a servlet is a strategy object that can be installed into any standard EE web server, and is the basis of two web view technologies: the older JavaServer Pages (JSP) and the newer, component-based JavaServer Faces (JSF). Allocation and injection of components is provided by the Contexts and Dependency Injection framework (CDI). Finally, EE provides a number of other network-based services, including EJB3 remote access and Java Messaging Service (JMS). These are outside the scope of this book; they are covered in other books, such as Arun Gupta's *Java EE 7 Essentials* (O'Reilly). This chapter is primarily for those who need or want to build their own server from the ground up.

15.1 Opening a Server Socket for Business

Problem

You need to write a socket-based server.

Solution

Create a `ServerSocket` for the given port number.

Discussion

The `ServerSocket` represents the *backend* (service end) of a connection, the server that waits patiently for clients to come along and connect to it. You construct a `ServerSocket` with just the port number.[1] Because it doesn't need to connect to another host, it doesn't need a particular host's address as the client socket constructor does.

Assuming the `ServerSocket` constructor doesn't throw an exception, you're in business. Your next step is to await client activity, which you do by calling `accept()`. This call blocks until a client connects to your server; at that point, the `accept()` returns to you a `Socket` object (not a `ServerSocket`) that is connected in both directions to the `Socket` object on the client (or its equivalent, if written in another language). Example 15-1 shows the code for a socket-based server.

Example 15-1. main/src/main/java/network/Listen.java

```
public class Listen {
  /** The TCP port for the service. */
  public static final short PORT = 9999;

  public static void main(String[] argv) throws IOException {
    ServerSocket sock;
    Socket  clientSock;
    try {
      sock = new ServerSocket(PORT);
      System.out.println("Listening on " + PORT);
      while ((clientSock = sock.accept()) != null) {

        // Process it, usually on a separate thread
        // to avoid blocking further accept() calls.
        process(clientSock);
```

```
      }

    } catch (IOException e) {
      System.err.println(e);
    }
  }

  /** This would do something with one client. */
  static void process(Socket s) throws IOException {
    System.out.println("Processing request from client " +
      s.getInetAddress());
    // The conversation would be here.
    s.close();
  }
}
```

You would normally use the same socket for both reading and writing, as shown in the next few recipes.

You may want to listen only on a particular network interface. Though we tend to think of network addresses as computer addresses, the two are not the same. A network address is actually the address of a particular network card, or network interface connection, on a given computing device. A desktop computer, laptop, tablet, or mobile phone might have only a single external interface (in addition to localhost/127.0.0.1), hence a single network address. But a large server machine might have two or more external interfaces, usually when it is connected to several networks. A network router is a box, either special purpose (e.g., a commercial internet router) or general purpose (e.g., a Unix/Linux host), that has interfaces on multiple networks *and* has both the capability and the administrative permission to forward packets from one network to another. A program running on such a server machine might want to provide services only to its inside network or its outside network. One way to accomplish this is by specifying the network interface to be listened on. Suppose you want to provide a different view of web pages for your intranet than you provide to outside customers. For security reasons, you probably wouldn't run both these services on the same machine. But if you wanted to, you could do this by providing the network interface addresses as arguments to the `ServerSocket` constructor.

However, to use this form of the constructor, you don't have the option of using a string for the network address's name, as you did with the client socket; you must convert it to an `InetAddress` object. You also have to provide a `backlog` argument, which is the

number of connection requests that can queue up to be accepted before clients are told that your server is too busy. The complete setup is shown in Example 15-2.

Example 15-2. main/src/main/java/network/ListenInside.java

```
public class ListenInside {
  /** The TCP port for the service. */
  public static final short PORT = 9999;
  /** The name of the network interface. */
  public static final String INSIDE_HOST = "acmewidgets-inside";
  /** The number of clients allowed to queue */
  public static final int BACKLOG = 10;

  public static void main(String[] argv) throws IOException {
    ServerSocket sock;
    Socket  clientSock;
    try {
      sock = new ServerSocket(PORT, BACKLOG,
        InetAddress.getByName(INSIDE_HOST));
      while ((clientSock = sock.accept()) != null) {

        // Process it.
        process(clientSock);
      }

    } catch (IOException e) {
      System.err.println(e);
    }
  }

  /** Hold server's conversation with one client. */
  static void process(Socket s) throws IOException {
    System.out.println("Connected from  " + INSIDE_HOST +
      ": " + s.getInetAddress(  ));
    // The conversation would be here.
    s.close();
  }
}
```

`InetAddress.getByName()`, described in Recipe 14.3, looks up the given hostname in a system-dependent way, typically referring to a configuration file in the */etc* or *\windows\System32\drivers\etc* directory, or to some kind of resolver such as the Domain Name System. Consult a good book on system and networking administration if you need to modify this data.

15.2 Finding Network Interfaces

Problem

You wish to find out about the computer's networking arrangements.

Solution

Use the `NetworkInterface` class.

Discussion

Every computer on a network has one or more *network interfaces*. On typical desktop machines, a network interface represents a network card or network port or some software network interface, such as the loopback interface. Each interface has an operating system–defined name. On most versions of Unix, these devices have a two- or three-character device driver name plus a digit (starting from 0), for example, `eth0` or `en0` for the first Ethernet on systems that hide the details of the card manufacturer, or `de0` and `de1` for the first and second Digital Equipment[2] DC21x4x-based Ethernet card, `xl0` for a 3Com EtherLink XL, and so on. The loopback interface is almost invariably `lo0` on all Unix-like platforms.

So what? Most of the time this is of no consequence to you. If you have only one network connection, like a cable link to your ISP, you really don't care. Where this matters is on a server, where you might need to find the address for a given network, for example. The `NetworkInterface` class lets you find this information. It has static methods for listing the interfaces and other methods for finding the addresses associated with a given interface. The program in Example 15-3 shows some examples of using this class. Running it prints the names of all the local interfaces. It then prints the machine's localhost network address and its associated interface.

Example 15-3. main/src/main/java/network/NetworkInterfaceDemo.java

```
public class NetworkInterfaceDemo {
  public static void main(String[] a) throws IOException {
    Enumeration<NetworkInterface> list = NetworkInterface.getNetworkInterfaces();
    while (list.hasMoreElements()) {
      // Get one NetworkInterface
      NetworkInterface iface = list.nextElement();
      // Print its name
```

```
        System.out.print(iface.getDisplayName());
        System.out.print(": ");
        Enumeration<InetAddress> addrs = iface.getInetAddresses();
        // And its address(es)
        while (addrs.hasMoreElements()) {
          InetAddress addr = addrs.nextElement();
          System.out.print(addr);
          System.out.print(' ');
        }
        System.out.println();

      }
      // Try to get the Interface for a given local (this machine's) address
      InetAddress destAddr = InetAddress.getByName("localhost");
      try {
        NetworkInterface dest = NetworkInterface.getByInetAddress(destAddr);
        System.out.println("Address for " + destAddr + " is " + dest);
      } catch (SocketException ex) {
        System.err.println("Couldn't get address for " + destAddr);
      }
    }
  }
```

15.3 Returning a Response (String or Binary)

Problem

You need to write a string, binary, or object data to the client.

Solution

The socket gives you an `InputStream` and an `OutputStream`. Use them.

Discussion

The client socket examples in the previous chapter called the `getInputStream()` and `getOutputStream()` methods. These examples do the same. The main difference is that these examples get the socket from a `ServerSocket`'s `accept()` method. Another distinction is that, by definition, the server normally creates or modifies the data and sends it to the client. Example 15-4 is a simple `Echo` server, which the `Echo` client of Recipe

14.5 can connect to. This server handles one complete connection with a client, then goes back and does the `accept()` to wait for the next client.

Example 15-4. main/src/main/java/network/EchoServer.java

```
public class EchoServer {
  /** Our server-side rendezvous socket */
  protected ServerSocket sock;
  /** The port number to use by default */
  public final static int ECHOPORT = 7;
  /** Flag to control debugging */
  protected boolean debug = true;

  /** main: construct and run */
  public static void main(String[] args) {
    int p = ECHOPORT;
    if (args.length == 1) {
      try {
        p = Integer.parseInt(args[0]);
      } catch (NumberFormatException e) {
        System.err.println("Usage: EchoServer [port#]");
        System.exit(1);
      }
    }
    new EchoServer(p).handle();
  }

  /** Construct an EchoServer on the given port number */
  public EchoServer(int port) {
    try {
      sock = new ServerSocket(port);
    } catch (IOException e) {
      System.err.println("I/O error in setup");
      System.err.println(e);
      System.exit(1);
    }
  }

  /** This handles the connections */
  protected void handle() {
    Socket ios = null;
    while (true) {
      try {
        System.out.println("Waiting for client...");
        ios = sock.accept();
```

```
        System.err.println("Accepted from " +
          ios.getInetAddress().getHostName());
        try (BufferedReader is = new BufferedReader(
            new InputStreamReader(ios.getInputStream(), "8859_1"));
          PrintWriter os = new PrintWriter(
            new OutputStreamWriter(ios.getOutputStream(), "8859_1"),
            true);) {
          String echoLine;
          while ((echoLine = is.readLine()) != null) {
            System.err.println("Read " + echoLine);
            os.print(echoLine + "\r\n");
            os.flush();
            System.err.println("Wrote " + echoLine);
          }
          System.err.println("All done!");
        }
      } catch (IOException e) {
        System.err.println(e);
      }
    }
  }
}
```

To send a string across an arbitrary network connection, some authorities recommend sending both the carriage return and the newline character; many protocol specifications require that you do so. This explains the `\r\n` in the code. If the other end is a DOS program or a Telnet-like program, it may be expecting both characters. On the other hand, if you are writing both ends, you can simply use `println()`—followed always by an explicit `flush()` before you read—to prevent the deadlock of having both ends trying to read with one end's data still in the `PrintWriter`'s buffer!

If you need to process binary data, use the data streams from `java.io` instead of the readers/writers. A server for the `DaytimeBinary` program of Recipe 14.6 should look like the following:

```
C:\javasrc\network>java network.DaytimeBinary
Remote time is 3161316799
BASE_DIFF is 2208988800
Time diff == 952284799
Time on localhost is Sun Mar 08 19:33:19 GMT 2014

C:\javasrc\network>time/t
Current time is  7:33:23.84p
```

```
C:\javasrc\network>date/t
Current date is Sun 03-08-2014

C:\javasrc\network>
```

Well, it happens that I have such a program in my arsenal, so I present it in Example 15-5. Note that it directly uses certain public constants defined in the client class. Normally these are defined in the server class and used by the client, but I wanted to present the client code first.

Example 15-5. main/src/main/java/network/DaytimeServer.java

```
public class DaytimeServer {
 /** Our server-side rendezvous socket */
 ServerSocket sock;
 /** The port number to use by default */
 public final static int PORT = 37;

 /** main: construct and run */
 public static void main(String[] argv) {
  new DaytimeServer(PORT).runService();
 }

 /** Construct a DaytimeServer on the given port number */
 public DaytimeServer(int port) {
  try {
   sock = new ServerSocket(port);
  } catch (IOException e) {
   System.err.println("I/O error in setup\n" + e);
   System.exit(1);
  }
 }

 /** This handles the connections */
 protected void runService() {
  Socket ios = null;
  DataOutputStream os = null;
  while (true) {
   try {
    System.out.println("Waiting for connection on port " + PORT);
    ios = sock.accept();
    System.err.println("Accepted from " +
     ios.getInetAddress().getHostName());
```

```
        os = new DataOutputStream(ios.getOutputStream());
        long time = System.currentTimeMillis();

        time /= 1000;  // Daytime Protocol is in seconds

        // Convert to Java time base.
        time += TimeClient.BASE_DIFF;

        // Write it, truncating cast to int since it is using
        // the Internet Daytime Protocol which uses 4 bytes.
        // This will fail in the year 2038, along with all
        // 32-bit timekeeping systems based from 1970.
        // Remember, you read about the Y2038 crisis here first!
        // (the above comment added to this program 2001-03-30).
        os.writeInt((int)time);
        os.close();
      } catch (IOException e) {
        System.err.println(e);
      }
    }
  }
}
```

The program in Example 14-8 in the previous chapter reads a `Date` object over an `ObjectInputStream`. Example 15-6, the `DaytimeObjectServer` (the other end of that process), is a program that constructs a `Date` object each time it's connected to and returns it to the client.

Example 15-6. main/src/main/java/network/DaytimeObjectServer.java

```
public class DaytimeObjectServer {
  /** The TCP port for the object time service. */
  public static final short TIME_PORT = 1951;

  public static void main(String[] argv) {
    ServerSocket sock;
    Socket  clientSock;
    try {
      sock = new ServerSocket(TIME_PORT);
      while ((clientSock = sock.accept()) != null) {
        System.out.println("Accept from " +
          clientSock.getInetAddress());
        ObjectOutputStream os = new ObjectOutputStream(
          clientSock.getOutputStream());
```

```
        // Construct and write the Object
        os.writeObject(LocalDateTime.now());

        os.close();
      }

    } catch (IOException e) {
      System.err.println(e);
    }
  }
}
```

15.4 Handling Multiple Clients

Problem

Your server needs to handle multiple clients.

Solution

Use a thread for each.

Discussion

In the C world, several mechanisms allow a server to handle multiple clients. One is to use a special system call `select()` or `poll()`, which notifies the server when any of a set of file/socket descriptors is ready to read, ready to write, or has an error. By including its rendezvous socket (equivalent to our `ServerSocket`) in this list, the C-based server can read from any of a number of clients in any order. Java does not provide this call, because it is not readily implementable on some Java platforms. Instead, Java uses the general-purpose `Thread` mechanism, as described in Chapter 11 (threads are now commonplace in many programming languages, though not always under that name). Each time the code accepts a new connection from the `ServerSocket`, it immediately constructs and starts a new thread object to process that client.[3]

The Java code to implement accepting on a socket is pretty simple, apart from having to catch `IOException`s:

```
/** Run the main loop of the Server. */
void runServer( ) {
    while (true) {
        try {
            Socket clntSock = sock.accept( );
            new Handler(clntSock).handle( );
        } catch(IOException e) {
            System.err.println(e);
        }
    }
}
```

To use a thread, you should implement `Runnable`. The `Handler` class creates a `Runnable` containing the socket, and submits it to a thread pool (see Recipe 11.1):

```
threadPool.submit(new Handler(clntSock));
```

But as written, `Handler` is constructed using the normal socket returned by `accept()` and normally calls the socket's `getInputStream()` and `getOutputStream()` methods and holds its conversation in the usual way. I'll present a full implementation, a threaded `Echo` client. First, a session showing it in use:

```
$ java network.EchoServerThreaded
EchoServerThreaded ready for connections.
Socket starting: Socket[addr=localhost/127.0.0.1,port=2117,localport=7]
Socket starting: Socket[addr=darian/192.168.1.50,port=13386,localport=7]
Socket starting: Socket[addr=darian/192.168.1.50,port=22162,localport=7]
Socket ENDED: Socket[addr=darian/192.168.1.50,port=22162,localport=7]
Socket ENDED: Socket[addr=darian/192.168.1.50,port=13386,localport=7]
Socket ENDED: Socket[addr=localhost/127.0.0.1,port=2117,localport=7]
```

Here I connected to the server once with my `EchoClient` program and, while still connected, called it up again (and again) with an operating system–provided Telnet client. The server communicated with all the clients concurrently, sending the messages from the first client back to the first client, and the data from the second client back to the second client, and so on. In short, it works. I ended the sessions with the end-of-file character in the program and used the normal disconnect mechanism from the Telnet client. Example 15-7 is the code for the server.

Example 15-7. main/src/main/java/network/EchoServerThreaded.java

```
public class EchoServerThreaded {

 final static ExecutorService threadPool =
  Executors.newVirtualThreadPerTaskExecutor();
 public static final int ECHOPORT = 2007;

 public static void main(String[] av) {
  new EchoServerThreaded().runServer();
 }

 public void runServer() {
  ServerSocket sock;
  Socket clientSocket;

  try {
   sock = new ServerSocket(ECHOPORT);

   System.out.println("EchoServerThreaded ready for connections.");

   /* Wait for a connection */
   while (true) {
    clientSocket = sock.accept();
    /* Create a thread to do the communication, and start it */
    new Handler(clientSocket).handle();
   }
  } catch (IOException e) {
   /* Crash the server if IO fails. Something bad has happened */
   System.err.println("Could not accept " + e);
   System.exit(1);
  }
 }

 /** A Thread subclass to handle one client conversation. */
 class Handler {
  final Socket sock;

  Handler(Socket s) {
   sock = s;
  }

  public void handle() {
   System.out.println("Socket starting: " + sock);
   threadPool.submit( () -> {
    try (BufferedReader is = new BufferedReader(
```

```
            new InputStreamReader(sock.getInputStream()));
          PrintStream os = new PrintStream(
            sock.getOutputStream(), true);) {
          String line;
          while ((line = is.readLine()) != null) {
            os.print(line + "\r\n");
            os.flush();
          }
          sock.close();
        } catch (IOException e) {
          System.out.println("IO Error on socket " + e);
          return;
        }
        System.out.println("Socket ENDED: " + sock);
      });
    }
  }
}
```

This version uses the recently minted virtual threads mechanism to hopefully cope with a lot of clients. All we have to do to use these threads is swap in the `Executors.newVirtualThreadPerTaskExecutor()` call as the first statement in the class. Using traditional `Thread`s would have involved significant overhead, with every connection creating a new threaded object and destroying it when done. If you need that for some reason, just use one of the other `static new...Executor()` methods in the `ExecutorService`.

The performance of the virtual threads pool approach is reasonably good. For a small number of concurrent users, you might use a thread pool using conventional threads. It is also possible to implement a server of this sort with NIO, the "new" (back in Java 1.4) I/O package. However, the code to do so is more involved than anything in this chapter, and it is fraught with issues. There are several good tutorials on the internet for the person who needs to explore the performance of using NIO to manage server connections.

15.5 Serving the HTTP Protocol

Problem

You want to serve up a protocol such as HTTP.

Solution

Create a `ServerSocket` and write some code that speaks the particular protocol. Or in Java 18+, use the standard `SimpleFileServer`. You could also use a Java-powered web server such as Apache Tomcat or an Enterprise Edition (Jakarta EE) server such as JBoss WildFly.

Discussion

You can implement your own HTTP protocol server for very simple applications, which we'll do here. For basic applications, on Java 18 and later, you can also use the JDK tool `jwebserver` or its main class `SimpleFileServer`. For any serious development, you want to use Jakarta EE; see the note at the beginning of this chapter.

This example just constructs a `ServerSocket` and listens on it. When connections come in, they are replied to using the HTTP protocol. So it is somewhat more involved than the simple `Echo` server presented in Recipe 15.3. However, it's not a complete web server; the filename in the request is ignored, and a standard message is always returned. This is thus a *very* simple web server; it follows only the bare minimum of the HTTP protocol needed to send its response back. For a real web server written in Java, get Tomcat from the Apache Tomcat website or use any of the Jakarta/Java EE application servers. The code shown in Example 15-8, however, is enough to understand how to build a simple server that responds to requests using a protocol.

Example 15-8. main/src/main/java/network/WebServer0.java

```
public class WebServer0 {
  public static final int HTTP_PORT = 8080;
  public static final String CRLF = "\r\n";
  ServerSocket serverSock;
  /** A link to the source of this program, used in error message */
  static final String VIEW_SOURCE_URL =
  "https://github.com/IanDarwin/javasrc/tree/master/main/src/main/java/network";

  /**
   * Main method, just creates a server and calls its runServer().
   */
  public static void main(String[] args) throws Exception {
    var port = args.length == 1 ?  Integer.parseInt(args[0]) : HTTP_PORT;
    WebServer0 w = new WebServer0(port);
    System.out.println(
      "DarwinSys Java WebServer0 listening on http://localhost:" + port + "/");
```

```
  w.runServer();    // never returns!!
}

WebServer0() throws IOException {
  this(HTTP_PORT);
}

WebServer0(int port) throws IOException {
  this(new ServerSocket(port));
}

WebServer0(ServerSocket serverSock) {
  this.serverSock = serverSock;
}

/** RunServer accepts connections and passes each one to a handler. */
public void runServer() throws Exception {
  while (true) {
    try {
      Socket us = serverSock.accept();
      handle(us);
    } catch(IOException e) {
      System.err.println(e);
      return;
    }

  }
}

/** handle() handles one conversation with a Web client.
 * This is the only part of the program that "knows" HTTP.
 */
public void handle(Socket s) {
  BufferedReader is;  // inputStream, from Viewer
  PrintWriter os;    // outputStream, to Viewer
  try {
    String from = s.getInetAddress().toString();
    System.out.println("Accepted connection from " + from);
    is = new BufferedReader(new InputStreamReader(s.getInputStream()));
    String request = is.readLine();
    System.out.printf("Request: %s from %s\n", request, from);

    os = new PrintWriter(s.getOutputStream(), true);
    os.print("HTTP/1.0 200 Here is your data" + CRLF);
    os.print("Content-type: text/html" + CRLF);
    os.print("Server-name: DarwinSys Non-functional Java WebServer0" + CRLF);
```

```
      final String replyTemplate = """
       <html>
       <head>
        <title>Not a real server.</title>
        <meta name='viewport' content='width=device-width, initial-scale=1'/>
       </head>
       <body>
       <h1>Welcome, %s, but...</h1>
       <p>You have reached a desktop machine
       that does not run a real Web server.
       <p>Please pick another system!</p>
       <p>Or view <a href='%s'>
       this WebServer0 source on github</a>
       (scroll down to WebServer0.java).</p>
       <hr/>
       <em>Java-based WebServer0</em><hr/>
       </body>
       </html>
       """;
      String reply = replyTemplate.formatted(from, VIEW_SOURCE_URL);
      os.print("Content-length: " + reply.length() + CRLF);
      os.print(CRLF);
      os.print(reply + CRLF);
      os.flush();
      s.close();
    } catch (IOException e) {
      System.out.println("IOException " + e);
    }
    return;
  }
}
```

18 Using the predefined web server is quite simple and handles the HTTP(S) protocol for you. To use it from the command line, use the `jwebserver` command. To use it programmatically, you need only specify the location where the files live. Example 15-9 contains the code to do this.

WARNING

This API's classes, based around `SimpleServer`, are in the *com.sun* namespace, meaning it is unsupported and subject to change. Further, since Oracle donated the Servlet API to the Eclipse Foundation as part of what is now Jakarta, they seem to have had to re-implement large parts of it, albeit with the `Exchange` class replacing the `HttpServletRequest` and `HttpServletResponse` classes.

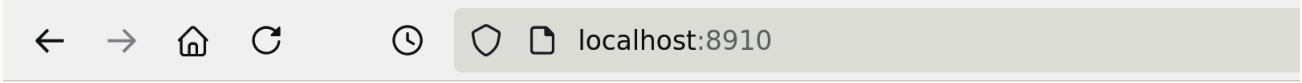
Example 15-9. main/src/main/java/netweb/SimpleServerDemo.java

```
void main() {
  var addr = new InetSocketAddress(PORT);
  var path = Path.of(System.getProperty("user.home") + "/public_html");
  var server = SimpleFileServer.createFileServer(addr, path, OutputLevel.INFO);
  System.out.printf("Starting listening on port %d to serve %s\n", PORT, path);
  server.start();
}
```

Running it looks simlar to running the hand-rolled version earlier in this recipe:

```
$ java netweb/SimpleServerDemo.java
Starting listening on port 8910 to serve /home/ian/public_html
127.0.0.1 - - [22/Jul/2024:14:08:24 -0400] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [22/Jul/2024:14:08:25 -0400] "GET /favicon.ico HTTP/1.1" 404 -
^C
```

The directory *~/public_html* was once commonly utilized—at the dawn of the web, before social media, almost everybody had their own web pages at that location on some server or another. A URL path beginning with *~ian* would then map to the *public_html* folder in my home directory. Figure 15-1 shows the output of the preceding code generated in a standard browser.



Ian's Demo Page

This is a dummy page just to show off the SimpleWebServer and other demos.

Figure 15-1. `SimpleServerDemo` in action

We can make it fancier by implementing the `Handler` portion of this server; this is shown in *main/src/main/java/netweb/SimpleServerDemo2.java*, which implements an `HttpHandler`, but doesn't have the code needed to map URLs to files. It instead looks at the request URL to check for bad-actor exploit attempts, and acts accordingly. The designers of the API would prefer that that kind of checking be done in their `Filter` class, but I wanted to keep the example short.

15.6 Securing a Web Server with TLS (formerly SSL) and JSSE

Problem

You want to protect your network traffic from prying eyes or malicious modification while the data is in transit.

Solution

Use the Java Secure Socket Extension (JSSE) to encrypt your traffic.

Discussion

JSSE provides services at a number of levels, but the easiest way to use it is simply to get your `ServerSocket` from an `SSLServerSocketFactory` instead of using the `ServerSocket` constructor directly. SSL is the Secure Sockets Layer; a revised version is known as TLS, Transport Layer Security. It is specifically for use on the web. To secure other protocols, you'd have to use a different form of the `SocketFactory`.

The `SSLServerSocketFactory` returns a `ServerSocket` that is set up to do SSL encryption. Example 15-10 uses this technique to override the `getServerSocket()` method in Recipe 15.5. If you're thinking this is too easy, you're wrong! The code is easy, but certificate management also requires work.

Example 15-10. main/src/main/java/network/JSSEWebServer0

```
/**
 * JSSEWebServer - subclass trivial WebServer0 to make it use SSL.
 * N.B. You MUST have set up a server certificate (see the
 * accompanying book text), or you will get the dreaded
 * javax.net.ssl.SSLHandshakeException: no cipher suites in common
 * (because without it JSSE can't use any of its built-in ciphers!).
 */
public class JSSEWebServer0 extends WebServer0 {

  public static final int HTTPS_PORT = 8443;

  public JSSEWebServer0() throws IOException {
    super();
  }

  public static void main(String[] args) throws Exception {
    if (System.getProperty("javax.net.ssl.keyStore") == null) {
      System.err.println(
        "You must pass in a keystore via -D; see the documentation!");
      System.exit(1);
    }
    System.out.println("DarwinSys JSSE Server 0.0 starting...");
    SSLServerSocketFactory ssf =
      (SSLServerSocketFactory)SSLServerSocketFactory.getDefault();
    var sock = ssf.createServerSocket(HTTPS_PORT);
    WebServer0 w = new WebServer0(sock);
    w.runServer();    // never returns!!
  }
}
```

That is, indeed, all the Java code one needs to write. You do have to set up an SSL certificate. For demonstration purposes, this can be a self-signed certificate; the steps are outlined on my website (steps 1–4 will suffice). You have to tell the JSSE layer where to find your keystore:

```
java -Djavax.net.ssl.keyStore=/home/ian/.keystore -Djavax.net.ssl.
keyStorePassword=secrit JSSEWebServer0
```

The typical client browser raises its eyebrows at a self-signed certificate (see Figure 15-2), but will accept the certificate if a user okays it. If you need a "real" certificate, check out Let's Encrypt, a nonprofit providing browser-accepted certificates at no cost.

Figure 15-3 shows the output of `WebServer0` being displayed over the HTTPS protocol (notice the padlock in the lower-right corner).

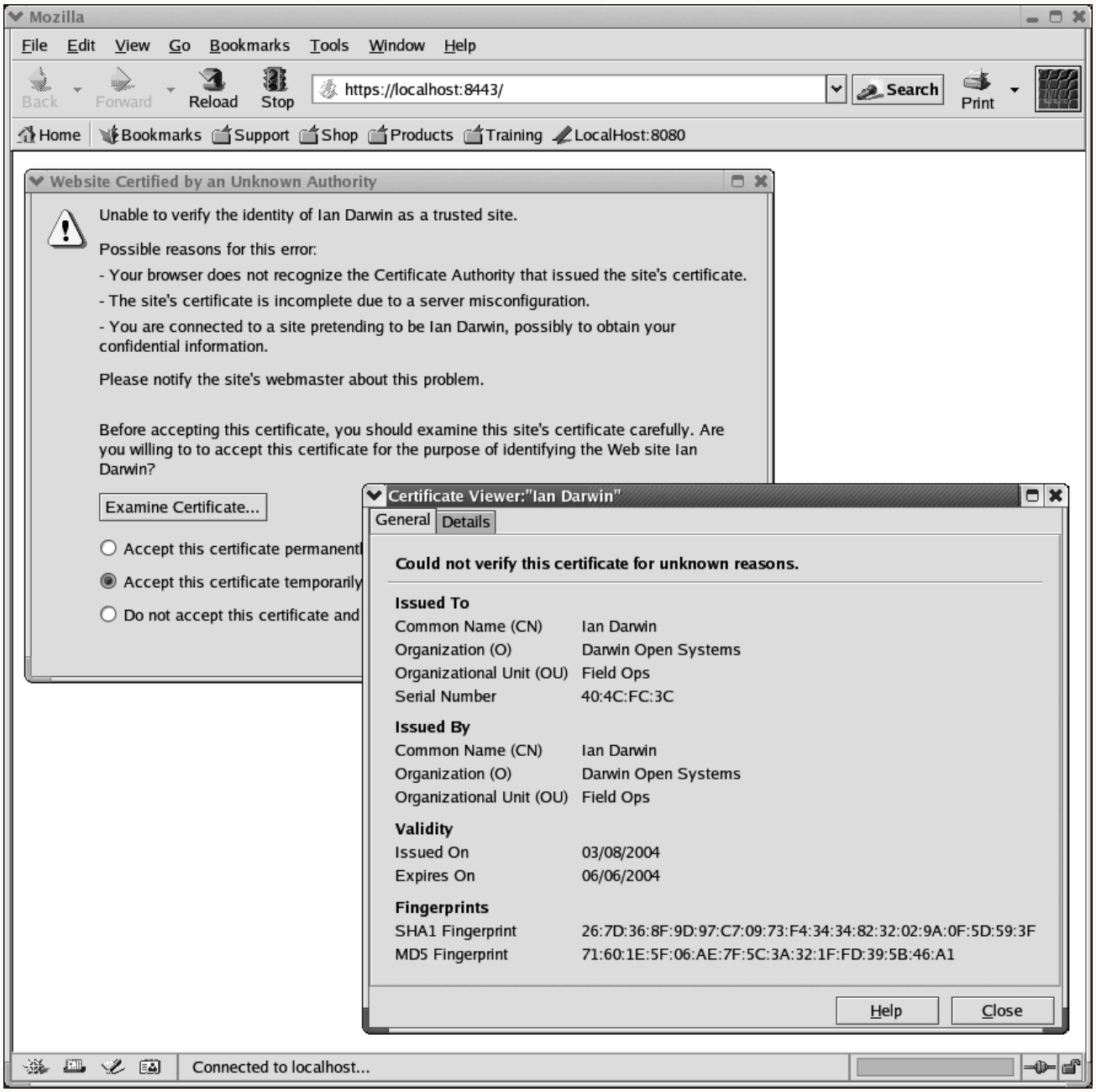


Figure 15-2. Browser caution

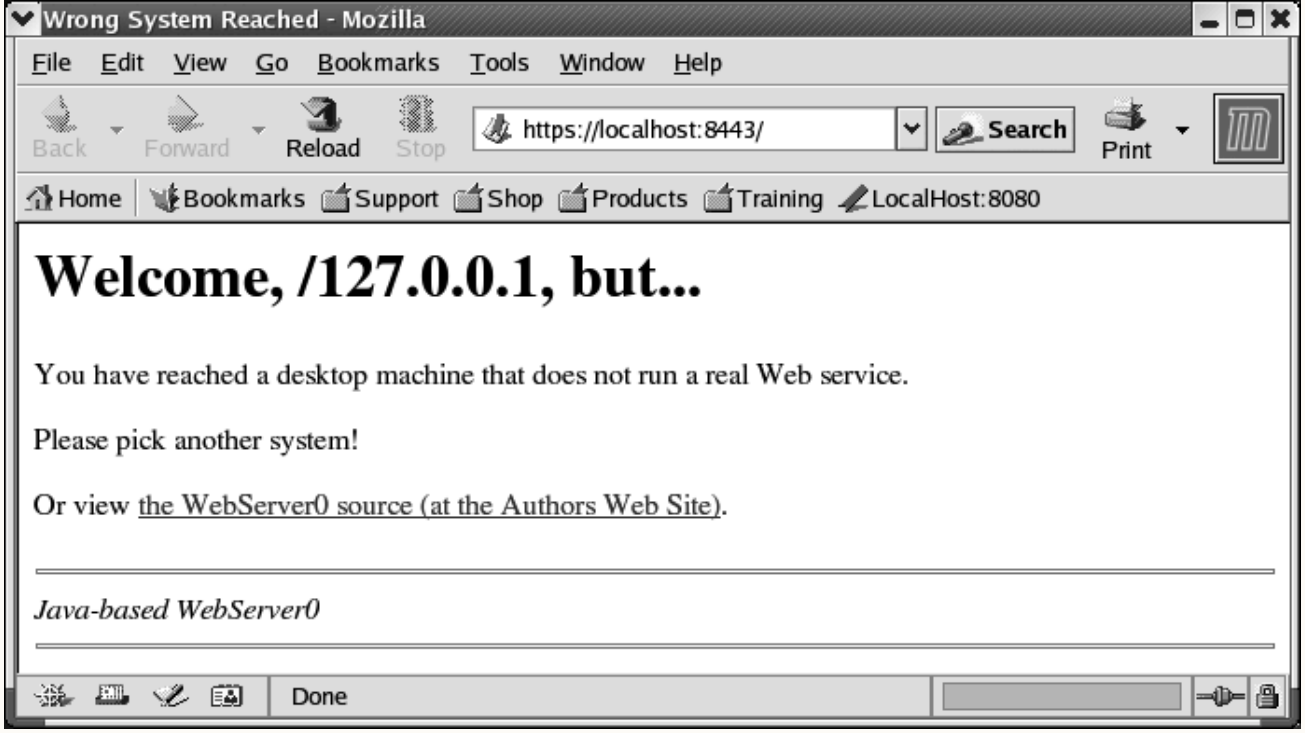


Figure 15-3. With encryption

See Also

JSSE can do much more than encrypt web server traffic; this is, however, sometimes seen as its most exciting application. For more information on JSSE, see the Oracle reference or *Java Security* by Scott Oaks (O'Reilly).

15.7 Creating a REST Service/Microservice with JAX-RS

Problem

You want to implement a RESTful server by using the provided Java EE/Jakarta EE APIs.

Solution

Use JAX-RS annotations on a class that provides a service; install it in an enterprise application server. Or use a framework that provides its own micro HTTP server.

Discussion

This operation consists of both coding and configuration. The latter can be done by some of the higher-level frameworks listed at the end of this recipe.

The coding steps consist of creating a class that extends the JAX-RS `Application` class and adding annotations to a "resource" class that provides a service.

Here is a basic `Application` class:

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;

@ApplicationPath("")
public class RestApplication extends Application {
      // Empty
}
```

Example 15-11 is a "Hello, World"-type service class with the annotations needed to make it a service class and to have three sample methods.

Example 15-11. restdemo/src/main/java/rest/RestService.java

```
@Path("")
@ApplicationScoped
public class RestService {

 public RestService() {
  System.out.println("RestService.init()");
 }

 @GET @Path("/timestamp")
 @Produces(MediaType.TEXT_PLAIN)
 public String getDate() {
  return LocalDateTime.now().toString();
 }

 /** A Hello message method
  */
 @GET @Path("/greeting/{userName}")
 @Produces("text/html")
 public String doGreeting(@PathParam("userName")String userName) {
  System.out.println("RestService.greeting()");
  if (userName == null || userName.trim().length() <= 3) {
   return "Missing or too-short username";
  }
  return String.format("<h1>Welcome %s</h1><p>%s, We're glad to see you back!",
   userName, userName);
 }
```

```
  /** Used to download all items */
  @GET @Path("/names")
  @Produces(MediaType.APPLICATION_JSON)
  public List<String> findTasksForUser() {
    return List.of("Robin", "Jedunkat", "Lyn", "Glen");
  }
}
```

Now the class must be deployed. If we have created a proper Maven project structure (see Recipe 2.4) and have provided an application-server-specific Maven plug-in, and our development server is running, we can use some variation on `mvn deploy`. In the present example I have set this up, in the *rest* subdirectory, for deployment to WildFly, a Java Enterprise server from the JBoss open source community, funded by Red Hat, Inc. I need only type `mvn wildfly:deploy` to have the application compiled, packaged, and deployed to my server.

Once the service is deployed, and prior to writing a proper client, you can explore it interactively with a browser or, for simple GET requests, a Telnet client or `netcat` / `nc`:

```
$ telnet localhost 8080
Escape character is '^]'.
GET /rest/timestamp HTTP/1.0
Connection: keep-alive

HTTP/1.1 200 OK
Content-Type: text/plain;charset=UTF-8

2025-10-16T19:54:31.42

GET /rest/greeting/Ian%20Darwin HTTP/1.0

HTTP/1.1 200 OK
Content-Type: text/html;charset=UTF-8

<h1>Welcome Ian Darwin</h1><p>Ian Darwin, We are glad to see you back!

get /rest/names HTTP/1.0
Accept: Application/JSON

HTTP/1.1 200 OK
Content-Type: application/json
```

```
["Robin","Jedunkat","Lyn","Glen"]
^] (CTRL/C)
$
```

An issue with REST is that there is not an official standard for documenting the API or protocol offered by a server (there are several competing specifications). People writing clients must either rely on plain documentation offered by the server's developers, or use trial and error to discover the protocol. Our example here is simple enough that we don't have this problem, but imagine a class with 20 or 30 methods in it.

REST frameworks

The two main midlevel toolkits that handle REST services (implementing JAX-RS) are RESTEasy and Jersey. Both are pretty much indistinguishable and are used by many of the higher-level REST frameworks. Another specification is Eclipse MicroProfile, a subset ("profile") of the Jakarta EE spec intended for lightweight REST services/microservices.

The term *microservice architecture* (MSA) has become popular. A microservice is a small, self-contained service that "owns its own data," that is, can perform operations without having to wait for responses from a separate database server. (It's assumed that mutable data will eventually migrate to a central database, but this will be done in the background.) There are numerous articles and books on MSA; the architectural concerns are a bit out of scope for this book. Take a look at the O'Reilly book *Head First Software Architecture* by Raju Gandhi, Mark Richards, and Neal Ford.

High level frameworks

The higher-level REST frameworks listed in Table 15-1 are free to use and reasonably popular. Each provides a complete framework for building and deploying REST services or microservices. That is, they both include the REST toolkit and provide a small, fast HTTP protocol server.

Table 15-1. REST/Microservice frameworks

Name	Home URL	Notes
Helidon	*https://helidon.io/*	Sponsored by Oracle
Quarkus	*https://quarkus.io*	Community framework
Micronaut	*https://micronaut.io/*	Community framework
Spring Boot	*https://spring.io/projects/spring-boot*	Plays well with other Spring packages

See Also

See this longer discussion on REST and the major frameworks (from 2020).

And if you're worried about the performance of Java versus other "cool new" languages, don't be. See this blog post from Mark Nelson that compares Java microservices to the Go language.

15.8 Unix Domain Sockets—Even on Windows! 16

Problem

You want to communicate more efficiently, and both the client and server are on the same machine.

Solution

Use the Unix domain socket mechanism.

Discussion

Unix domain sockets are an optimization for same-system networking. They were introduced into BSD Unix around 1979, but only appeared in Microsoft Windows in the 2010s. At the C programming level they work largely like network sockets. In Java they are best accessed with NIO buffers, whereas regular sockets can work directly with Strings. Otherwise their behavior is similar. They are especially useful in communication between an application server and its database; these often run on the same machine or VM.

This recipe shows both the client and the server because they share some code. The program is a simple server example, which sends a fixed string from the client to the server. Example 15-12 is the server portion.

Example 15-12. main/src/main/java/network/UnixDomainSocketServer.java

```
System.out.println("Unix Domain Sockets Demo");
Path socketPath = Path.of("/tmp/.jcb.socket");

System.out.println("Server side");
ServerSocketChannel serverChan =                    ❶
 ServerSocketChannel.open(StandardProtocolFamily.UNIX);
UnixDomainSocketAddress sockAddr =                  ❷
 UnixDomainSocketAddress.of(socketPath);
serverChan.bind(sockAddr);                          ❸
System.out.println("Waiting...");
SocketChannel channel;
while ((channel = serverChan.accept()) != null) {   ❹
 ByteBuffer inBuf = ByteBuffer.allocate(BUFSIZE);   ❺
 int numBytes;
 while ((numBytes = channel.read(inBuf)) > 0) {     ❻
  byte[] bytes = new byte[numBytes];
  inBuf.flip();                                     ❼
  inBuf.get(bytes);
  String message = new String(bytes);               ❽
  System.out.printf("[Incoming] %s\n", message);
  inBuf.clear();
 }
}
```

❶ Create the `ServerSocket` to accept connections from clients.

❷ Set up the IP address that clients will connect to.

❸ Hook up the IP address to the `ServerSocket`.

❹ Accept a connection from the client.

❺ Set up a buffer (chunk of memory) to read data into.

❻ Do the actual read, return number of bytes actually read, and quit the loop if we get no data (implies normal end-of-file condition).

❼ Create our own buffer to hold the data, and move the NIO buffer to a state where we can fetch from it.

❽ Create a `String` containing the bytes we read, and print it.

The corresponding client sender is shown in Example 15-13.

Example 15-13. main/src/main/java/network/UnixDomainSocketClient.java

```
System.out.println("Unix Domain Sockets Demo");
Path socketPath = Path.of("/tmp/.jcb.socket");

System.out.println("Client Starting");
UnixDomainSocketAddress sockAddr = UnixDomainSocketAddress.of(socketPath);
String message = "Hello via a UNIX Domain Socket";
try (SocketChannel channel = SocketChannel.open(StandardProtocolFamily.UNIX)) {
  channel.connect(sockAddr);
  ByteBuffer outBuf = ByteBuffer.allocate(BUFSIZE);
  outBuf.clear();
  outBuf.put(message.getBytes());
  outBuf.flip();
  System.out.printf("[Sending] %s\n", message);
  while (outBuf.hasRemaining()) {
    channel.write(outBuf);
  }
} catch (Exception ex) {
  throw new RuntimeException("Send failed: " + ex, ex);
}
```

When we run both programs:

```
$ java UnixDomainSocketServer.java &
[1] 42962
Unix Domain Sockets Demo
Server side
Waiting...
$ java UnixDomainSocketClient.java
Unix Domain Sockets Demo
Client Starting
[Sending] Hello via a UNIX Domain Socket
[Incoming] Hello via a UNIX Domain Socket
$
```

See Also

A good general reference on this chapter's topic is *Java Network Programming* by Elliotte Rusty Harold (O'Reilly).

The server side of any network mechanism is extremely sensitive to security issues. It is easy for one misconfigured or poorly written server program to compromise the security of an entire network! Of the many books on network security, two stand out: *Firewalls and Internet Security* by William R. Cheswick et al. (Addison-Wesley) and a series of books with *Hacking Exposed* in the title; the first in the series is by Stuart McClure et al. (McGraw-Hill).

This completes my discussion of server-side Java using sockets. A chat server could be implemented using several technologies, such as Remote Methods Invocation (RMI), an HTTP web service, Java Message Service (JMS), and a Java Enterprise API that handles store-and-forward message processing. This is beyond the scope of this book, but there's an example of an RMI chat server in the *chat* folder of the source distribution, and there's an example of a JMS chat server in *Java Message Service* by Mark Richards et al. (O'Reilly).

[1] You may not be able to pick just any port number for your own service, of course. Certain well-known port numbers are reserved for specific services and listed in your *services* file, such as 22 for Secure Shell and 25 for SMTP. Also, on server-based operating systems, ports below 1024 are considered privileged ports and require root or administrator privilege to create. This was an early security mechanism; today, with zillions of single-user desktops connected to the internet, it provides little real security, but the restriction remains.

[2] Digital Equipment was absorbed by Compaq, which was then absorbed by HP, but the name remains `de` because the engineers who name such things don't care for corporate mergers anyway.

[3] There are some limits to how many regular threads you can have running; if you expect to have thousands of threads running, particularly in a networked application, consider using virtual threads (see Recipe 11.2).

Previous chapter

< 14. Network Clients

Next chapter

16. Processing JSON Data >